

Algorithme du pagerank

Equipe K12
RAGOT Cyrian et MIELCAREK Titouan

Janvier 2024



Table des matières

1	Contenu du rapport	3
2	Introduction	3
3	Principe de l’algorithme	4
3.1	Architecture de l’application	4
3.2	Mise au point des modules	5
3.3	Choix réalisés concernant les structures de données et les algorithmes . .	6
4	Execution du programme	8
5	Difficultés rencontrées	9
6	Grilles d’évaluation	10
7	Conclusion	11
8	Annexe	11

1 Contenu du rapport

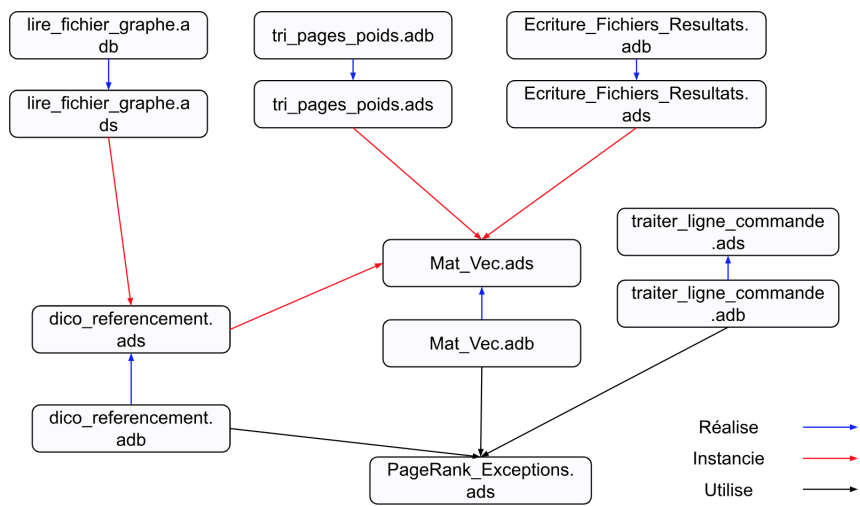
Ce rapport documente notre projet de mise en œuvre de l'algorithme de PageRank, visant à mesurer la popularité des pages Internet. Il détaillera tout d'abord le principe de fonctionnement de l'algorithme, les résultats de l'exécution ainsi que les difficultés rencontrées. Une auto évaluation des raffinages et du programme sera fournie à l'aide de grilles adaptées.

2 Introduction

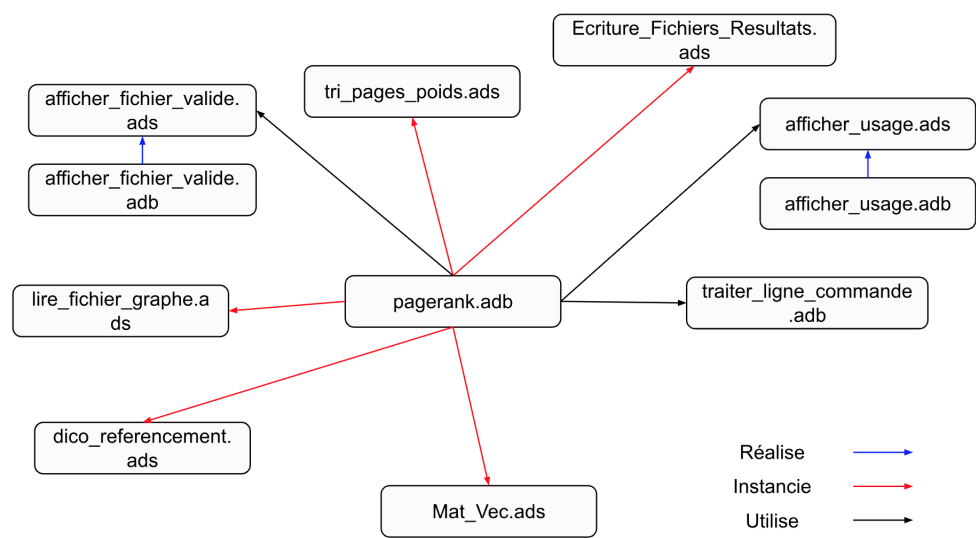
Le concept du PageRank repose sur une analogie avec les lettres de recommandation lors d'une recherche d'emploi. Une page Internet devient "reconnue" si elle est référencée par d'autres pages tout aussi reconnues, créant ainsi une hiérarchie basée sur la reconnaissance mutuelle. Notre démarche consiste à développer deux implémentations de l'algorithme de PageRank, chacune exploitant des approches distinctes, et à évaluer leurs performances de manière comparative. L'utilisateur doit pouvoir dialoguer avec l'algorithme via des arguments de ligne de commande.

3 Principe de l’algorithme

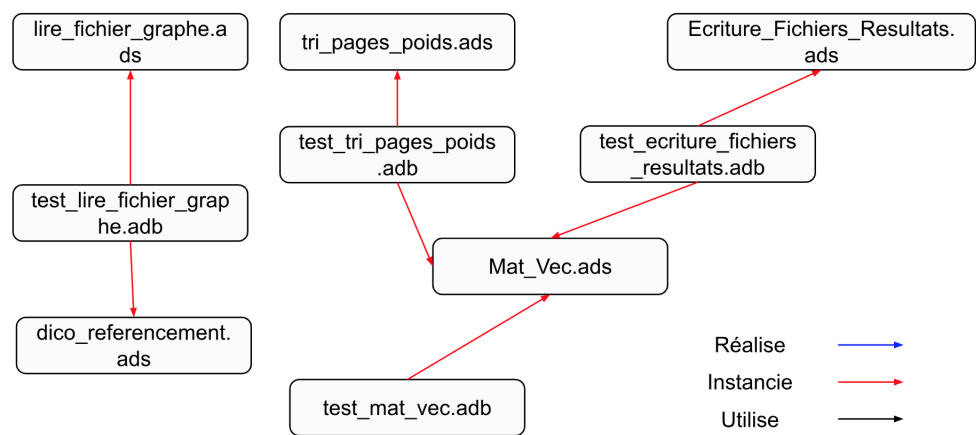
3.1 Architecture de l’application



Architecture des modules



Architecture du programme principal



Architecture des programmes de test

3.2 Mise au point des modules

Pour ce projet, nous avons fait le choix de maximiser l'utilisation de fichiers annexes pour définir différents modules et sous-programmes. Ainsi, le code est plus clair, lisible et compréhensible.

Faisons un bref aperçu de l'utilité de chaque module et sous-programmes.

L'interface *pagerank_exception* nous permet de spécifier les exceptions que l'on a défini : *Fichier_Graphe_Exception*, *Argument_Exception* et *Index_Exception*.

Les sous-programmes *afficher_fichier_valide* et *afficher_usage* permettent d'écrire dans le terminal des informations concernant la forme d'un fichier graphe valide et l'utilisation des options du pagerank. Ils sont utilisés pour expliquer à l'utilisateur les bonnes pratiques lorsqu'il utilise mal le programme.

Le sous-programme *tri_pages_poids* implémente le tri rapide utilisé pour trier le vecteur des poids et le vecteur des pages dans l'ordre décroissant selon les poids calculés dans le vecteur poids.

Le sous-programme *lire_fichier_graphe* permet de récupérer les liens présents dans le fichier graphe et de les enregistrer dans notre dictionnaire de référencement. Il lève l'exception *Fichier_Graphe_Exception* lorsque l'utilisateur utilise un fichier graphe qui n'est pas conforme.

Le sous-programme *traiter_ligne_commande* est utilisé pour lire la ligne de commande et les options fournies par l'utilisateur. Il lève l'exception *Argument_Exception* lorsque l'utilisateur n'a pas correctement écrit la ligne de commande.

Le module *ecriture_fichiers_resultats* implémente deux procédures qui permettent d'écrire les fichiers résultats en .pr et .prw avec les vecteurs triés avant.

Le module *dico_referencement* implémente le type de dictionnaire de référencement qui enregistre les liens entre les pages et ses procédures associées.

Le module *mat_vec* implémente les types de vecteur réel, vecteur entier, matrice pleine et matrice creuse et leurs procédures associées.

Enfin, nous avons écrit 5 programmes de tests unitaires pour tester nos modules : *test_dico_referencement*, *test_lire_fichier_graphe*, *test_mat_vec*, *test_lire_fichier_graphe* et *test_tri_pages_poids*.

Modules/Procédures	Spécifier	Programmer	Tester	Relire
afficher_fichier_valide	/	Cyrian	Cyrian	Titouan
afficher_usage	/	Cyrian	Cyrian	Titouan
dico_referencement	/	Titouan,Cyrian	Titouan,Cyrian	Cyrian
ecriture_fichiers_resultats	/	Cyrian	Cyrian	Titouan
lire_fichier_graphe	/	Cyrian	Titouan,Cyrian	Titouan
mat_vec	/	Titouan,Cyrian	Titouan,Cyrian	Cyrian
pagerank_exceptions	Titouan,Cyrian	/	/	Cyrian
traiter_ligne_commande	/	Cyrian	Cyrian	Titouan
tri_pages_poids	Cyrian	Cyrian	Cyrian	Titouan
test_dico_referencement	/	Titouan,Cyrian	/	Titouan
test_lire_fichier_graphe	/	Cyrian	/	Titouan
test_mat_vec	/	Titouan,Cyrian	/	Titouan
test_lire_fichier_graphe	/	Cyrian	/	Titouan
test_tri_pages_poids	/	Cyrian	/	Titouan
pagerank	Cyrian,Titouan	Cyrian	Cyrian	Titouan

3.3 Choix réalisés concernant les structures de données et les algorithmes

Nous avons plusieurs structures de données spécifiques dans notre projet.

Nous avons déjà implémenté un type de dictionnaire qui est un tableau de taille le nombre de pages du graphe. Chaque case correspond à une clé, son index correspond à une page. Chaque case contient la valeur associée à la clé qui est un enregistrement possédant une SDA des pages référencées par la page clé, ainsi que le nombre de pages référencées par la page clé. Ces données étant fréquemment utilisées dans le programme principal, il nous semblait important de faciliter leur accès. De plus, cette structure permet une bonne compréhension du graphe pour écrire la suite du programme et permet d'enlever les doublons directement.

Listing 1 – Types définis dans dico_referencement

```
type T_Cellule ;

type TLCA is access T_Cellule ;

type T_Cellule is
    record
        Page : Integer ;
        Suivant : TLCA ;
    end record ;

type T_Valeur is
    record
        Nb_Liens : Integer ;
        Pages_Ref : TLCA ;
    end record ;

type T_Dico_Referencement is array (0..Nombre_Pages-1) of T_Valeur ;
```

Deuxièmement, dans le module *mat_vec* nous implémentons les types de vecteur réels et d'entiers qui sont des simples tableaux de la taille du nombre de pages du graphe. Les matrices sont représentées par un type de matrice qui prend en argument un booléen permettant d'indiquer si on travaille avec des matrices creuses ou des matrices pleines. Lorsque ce type matrice est une matrice pleine, c'est un tableau carré de réels de taille le nombre de pages. Lorsque ce type matrice est une matrice creuse, c'est un tableau de taille le nombre de pages contenant des vecteurs creux qui sont nos lignes de notre matrice. Ce type de vecteurs creux est aussi un type bien défini qui utilise des procédures internes au module.

Listing 2 – Types définis dans dico_referencement

```
— vecteur de reels
type T_Vecteur_Reel is array (0..(Taille - 1)) of Long_Float ;

— vecteur d'entiers
type T_Vecteur_Entier is array (0..(Taille - 1)) of Integer ;

— matrice pleine
type T_Matrice_Pleine is array (0..(Taille - 1), 0..(Taille - 1))
    of Long_Float ;

— cellule des vecteurs creux
type T_Cellule ;
```

```

— vecteur creux
type T_Vecteur_Creux is access T_Cellule;

type T_Cellule is
    record
        Indice : Integer;
        Valeur : Long_Float;
        Suivant : T_Vecteur_Creux;
    end record;

— matrice creuse
type T_Matrice_Creuse is array (0..(Taille-1)) of T_Vecteur_Creux;

— matrice de reels
type T_Matrice(Pleine: Boolean) is
    record
        case Pleine is
            when True => Matrice_Pleine : T_Matrice_Pleine;
            when False => Matrice_Creuse : T_Matrice_Creuse;
        end case;
    end record;

```

Ensuite, pour les choix algorithmiques intéressants, nous avons tout d’abord le tri qui est un tri rapide (quick sort). Cet algorithme de tri est parmi les plus performants, en effet en terme de complexité moyenne il est en $O(n \log(n))$ mais reste en complexité $O(n^2)$ dans le pire cas.

Puis, un point intéressant à relever est le calcul du produit entre un vecteur horizontal et une matrice dans le cas de la matrice creuse. Nous avons choisi de calculer ce produit en parcourant chaque ligne de la matrice qui correspond donc à un vecteur creux, cela nous permettant de parcourir les éléments d’un vecteur creux d’un seul coup. Ce produit matriciel s’oppose à la méthode classique qui consiste à parcourir chaque colonne une à une et non pas les lignes une à une de la matrice. Cette méthode est sensée accélérer le calcul.

Enfin, nous avons créé un programme robuste au niveau à la fois de la ligne de commande et du fichier graphe utilisé. Cette robustesse a été implémentée via la levée d’exceptions que nous avons définis et par l’affichage d’aide à l’utilisateur lorsque ces exceptions sont attrapées dans le programme principal pagerank. Nous avons alors fait des essais sur plusieurs cas spéciaux pour vérifier cette robustesse.

4 Execution du programme

Nous avons testé les différents exemples de graphe sur une machine de l'école pour pouvoir comparer leurs temps d'exécution. De plus, avant de lancer nos essais, nous avons fixé la limite souple de l'utilisateur en illimité pour assurer de meilleures performances avec la commande *ulimit -s unlimited*. Les essais ont été effectués avec l'algorithme des matrices creuses et l'algorithme des matrices pleines pour les options par défaut de l'algorithme du pagerank (voir les tableaux suivants).

Graphe	Noeuds	Temps d'exécution
sujet.net	6	0m 0.024s
worm.net	279	0m 0.068s
brainlinks.net	10 000	2m 58s
linux26.net	285 510	Erreur de stockage

TABLE 1 – *Temps d'exécution pour différents graphes avec l'algorithme des matrices pleines*

Graphe	Taille	Temps d'exécution
sujet.net	6	0m 0.013s
worm.net	279	0m 0.084s
brainlinks.net	10 000	2m 5s
linux26.net	285 510	Estimé : 25h

TABLE 2 – *Temps d'exécution pour différents graphes avec l'algorithme des matrices creuses*

On observe que le calcul à l'aide de matrices creuses est effectivement plus efficace pour un grand graphe comme brainlinks.net. Cependant, worm.net est plus lent avec les matrices creuses, mais cela ne se joue à la centième de seconde donc on peut considérer que ce n'est pas pertinent de les comparer.

Ensuite, un grand avantage des matrices creuses est que l'on peut lancer le programme avec le plus grand graphe linux26.net alors qu'avec des matrices pleines le programme s'arrête immédiatement avec une erreur de mémoire. Malheureusement, le temps d'exécution pour ce graphe est très grand et en observant l'évolution du calcul à chaque itération nous avons seulement pu estimer le temps nécessaire qu'il faudrait pour terminer les 150 itérations avec linux26.net.

Nous avons tenté d'utiliser l'outil de profiling gprof sur notre programme mais celui-ci ne s'est pas avéré utile car il fournissait un résultat étrange dont nous ne pouvions pas tirer beaucoup d'informations et après échange avec quelques camarades, nous avons appris que cet outil ne fonctionne pas toujours bien selon la structure du programme. Alors, pour tenter de comprendre grossièrement les sous-programmes ou les parties du programme qui pouvaient être plus lentes, nous avons affiché dans le terminal des indicateurs lors de chaque opération. Nous avons pu comprendre que les actions qui nous faisaient perdre de la rapidité étaient les itérations du calcul des poids, chaque itération prenant un temps assez long.

5 Difficultés rencontrées

Voici quelques difficultés auxquelles nous avons été confrontés lors du projet.

Premièrement, il a été laborieux pour Titouan d'utiliser le compilateur gnat et git sur son ordinateur personnel qui posait des soucis et il a donc été forcé de travailler en parallèle sur les ordinateurs de l'école et cela nous a aussi posé des problèmes de version sur gitlab.

De plus, au départ nous avons créé un seul type de vecteur de réels pour les poids mais il nous fallait un vecteur d'entiers pour y mettre les pages et les trier dans ce vecteur donc nous avons choisi de créer un autre type vecteur d'entiers et non pas un vecteur qui prend un type générique car cela nous aurait obligé à redéfinir des fonctions pour faire des opérations élémentaires sur ce type générique, ce qui complique le développement pour peu puisque l'on est sûr d'avoir besoin que de ces deux types de vecteurs.

Ensuite, nous voulions initialement récupérer toutes les informations du fichier graphe (le nombre de pages et les liens) en une seule fois avec un module `lire_fichier_graphe` mais pour récupérer les liens et les retenir dans notre dictionnaire de référencement défini par le module `dico_referencement` nous avons besoin d'instancier ce module avec le bon nombre de pages qui est un paramètre de généricité. Au final, pour gérer ce problème, nous avons choisi de récupérer le nombre de pages dans le programme principal `page-rank` sans le module `lire_fichier_graphe` puis d'utiliser la syntaxe *declare* pour instancier tous les modules qui ont besoin ce nombre de pages en paramètre de généricité.

Un autre point qui a dû subir des changements est le calcul de l'itération du pagerank qui, en premier lieu, aurait dû utiliser une procédure *Reference* définie dans le module `dico_referencement` qui indique si une page en référence une autre dans un dictionnaire de référencement. Au final, le remplissage d'une matrice se fait directement dans le module `dico_referencement` dans lequel on a accès de manière plus simple aux types définis avec *limitedprivate*. Mais ceci nous a obligé à fournir le module `mat_vec` au module `dico_referencement` de manière générique, ce qui n'est pas très grave mais quelque peu confus.

Un dernier problème important (lié au problème précédent) a été que nous avons fait le choix de garder tous nos types en *limitedprivate* dans nos modules. Ainsi, le programme a été plus fastidieux à écrire sur certains points.

Finalement, nous avons eu plusieurs autres problèmes mineurs tout au long du projet mais par soucis de synthétisation il n'est pas pertinent d'en discuter d'avantage.

6 Grilles d'évaluation

		Evaluation Etudiant (I/P/A/+)
Forme (D-21)	Respect de la syntaxe Ri : Comment "... une action complexe ..."? des actions combinées avec des structures de contrôle Rj : ...	
	Verbe à l'infinitif pour les actions complexes	+
	Nom ou équivalent pour expressions complexes	+
	Tous les Ri sont écrits contre la marge et espacés	+
	Les flots de données sont définis	A
	Une seule décision ou répétition par raffinage	+
	Pas trop d'actions dans un raffinage (moins de 6)	+
	Bonne présentation des structures de contrôle	+
Fond (D21-D22)	Le vocabulaire est précis	A
	Le raffinage d'une action décrit complètement cette action	+
	Le raffinage d'une action ne décrit que cette action	+
	Les flots de données sont cohérents	+
	Pas de structure de contrôle déguisée	+
	Qualité des actions complexes	+

Etudiant (O/N)	Règle
O	Le programme ne doit pas contenir d'erreurs de compilation.
O	Le programme doit compiler sans messages d'avertissement.
O	Le code doit être bien indenté.
O	Les règles de programmation du cours doivent être respectées : toujours un Sinon pour un Si, pas de sortie au milieu d'une répétition...
O	Pas de code redondant.
O	On doit utiliser les structures de contrôle adaptées (Si/Selon/TantQue/Répéter/Pour)
O	Utiliser des constantes nommées plutôt que des constantes littérales.
O	Les raffinages doivent être respectés dans le programme.
O	Les actions complexes doivent apparaître sous forme de commentaires placés AVANT les instructions correspondantes, avec la même indentation
O	Une ligne blanche doit séparer les principales actions complexes
O	Le rôle des variables doit être explicité à leur déclaration (commentaire).

7 Conclusion

Pour conclure, notre programme pagerank permet de trier les pages selon leur popularité, popularité qui se définit à partir des liens entre les pages. Les fichiers sujet.net, worm.net et brainlinks.net se lancent avec les matrices pleines mais le fichier linux26.net ne peut être lancé dans ces conditions. Ceci est dû au fait que des matrices d’une telle taille occupent trop d’espace mémoire. L’utilisation de matrices creuses permet de pallier à ce problème. Toutefois ce programme est lent en utilisant linux26.net, nonobstant la condensation des données apportées par l’utilisation des matrices creuses.

Pour aller plus loin nous aurions pu permettre à l’utilisateur de choisir un niveau de précision. Le niveau de précision étant un entier définissant le nombre de décimales. Le programme s’adapterait alors pour faire le calcul avec cette nouvelle précision. Actuellement, les réels sont des *LongFloat* pour avoir une bonne précision, mais celle-ci pourrait être adapté dans tous nos sous-programmes en implémentant la précision modifiable.

Aussi, nous avons remarqué que notre programme n’est pas très performant, plusieurs changements pourraient être effectués pour tenter de l’améliorer, surtout au niveau des calculs effectués dans l’itération du calcul des poids. Par exemple, une autre approche considérant les vecteurs creux comme des colonnes et non plus comme des lignes pour former la matrice creuse, pourrait éviter des calculs complexes lors du produit vecteur-matrice. Et avec plus de temps nous aurions pu tenter de cibler quelles opérations sont plus coûteuses.

La robustesse du programme pourrait certainement aussi être améliorée car certains cas d’utilisation n’ont pu être pris en compte.

8 Annexe

De ce projet, nous avons tiré de la connaissance sur le travail en équipe, notamment le fait d’utiliser des noms de programmes et variables clairs afin de faciliter la communication, ainsi que la nécessité d’uniformiser les notations et la mise en forme. Notre vision commune d’un programme divisé en un maximum de module a permis d’éviter les désaccords.

Concernant les raffinages, nous avons une base lors du premier rendu qui nous a permis de faciliter le début de l’écriture du programme. Cependant, notre programme ayant bien dérivé des raffinages de base, nous avons réécrit les raffinages pour les adapter au programme. Ces raffinages ont été écrits de manière simple sans trop décomposer d’actions complexes pour le rendre lisible, surtout quand ces actions complexes nous semblait suffisamment simples avec les modules créés.